GLOSARIO

A - D

ACID: Propiedades de las transacciones en bases de datos: Atomicidad, Consistencia, Aislamiento, Durabilidad.

Acoplamiento: es la medida cuantitativa y cualitativa del grado de interdependencia entre los módulos, clases o componentes de un sistema informático. Determina el nivel de conocimiento que un módulo tiene sobre la estructura interna, el estado o la implementación de otro.

Activo digital: es cualquier elemento o recurso que existe en formato electrónico y tiene un valor económico, comercial o de uso.

Actuator: Módulo que expone endpoints para monitorizar la aplicación: salud, métricas, info, logs.

Administrador: Rol con permisos elevados dentro del sistema. Puede gestionar usuarios, activos digitales y acceder a vistas restringidas.

Application Context: El contenedor de IoC de Spring que centraliza, inicializa y configura todos los componentes de la aplicación.

application.properties / application.yml: Archivos de configuración centralizados donde se definen puertos, credenciales de bases de datos y variables del entorno.

Auditor: Usuario con permisos especiales para revisar, supervisar y validar las operaciones realizadas en el sistema. No modifica datos; solo observa y analiza.

Auditoría: Proceso de registro, supervisión y análisis de todas las acciones realizadas sobre los activos digitales. Garantiza trazabilidad, seguridad y control interno.

Autenticación: Proceso mediante el cual un usuario introduce su nombre y contraseña para acceder a la aplicación.

Backend: Parte lógica y servidor de una aplicación que procesa los datos y gestiona la base de datos.

Backup: (o copia de seguridad) es el proceso de duplicar y almacenar datos informáticos en un soporte secundario con el objetivo de poder recuperarlos en caso de pérdida, corrupción, sabotaje o fallo del sistema original.

Base de Datos H2: Sistema de almacenamiento embebido que guarda la información del MVP sin necesidad de instalar MySQL.

Beans: Objetos de Java que son creados, gestionados y ensamblados por el contenedor de Spring IoC.

Contenedor Docker (Docker Container): Instancia en ejecución de una imagen. Es la aplicación funcionando realmente dentro de un entorno aislado.

Control de Caché / Bloqueo de Navegación Atrás: Mecanismo que impide que el navegador muestre páginas protegidas después del logout. Se logra mediante cabeceras HTTP que deshabilitan el almacenamiento en caché.

Control Interno: Conjunto de mecanismos que garantizan que las operaciones del sistema se realizan de forma segura, transparente y conforme a las políticas establecidas.

Controller Layer / Service Layer / Repository Layer: Arquitectura por capas típica en Spring Boot:

Controller: recibe peticiones
Service: lógica de negocio
Repository: acceso a datos

CRUD: es un acrónimo en inglés que significa Create (Crear), Read (Leer), Update (Actualizar) y Delete (Borrar). Representa las cuatro operaciones básicas que se pueden realizar sobre cualquier información o datos almacenados de forma persistente en una base de datos o aplicación.

Dashboard: Panel principal del sistema donde el usuario accede a las funcionalidades clave: gestión de activos, perfil, logout, etc.

DevTools: Herramientas para desarrollo: reinicio automático, LiveReload, mejoras de productividad.

DTO (Data Transfer Object): Objeto utilizado para transportar datos desde el backend hacia la vista. En este módulo, el DTO permite mostrar nombres de usuarios y activos en lugar de IDs.

Docker build: Comando que construye una imagen a partir de un Dockerfile. Ejemplo: docker build -t zeerty-app:latest .

docker-compose: Herramienta que permite definir y ejecutar varios contenedores juntos mediante un archivo docker-compose.yml. Ideal para aplicaciones con varios servicios (app + base de datos).

docker compose down: Detiene y elimina los contenedores definidos en el compose.

docker compose up: Levanta todos los servicios definidos en docker-compose.yml.

docker-compose.yml: Archivo donde se describen los servicios, redes y volúmenes que forman la aplicación. Ejemplo: contenedor Spring Boot + contenedor MySQL.

Docker run: Comando que ejecuta un contenedor a partir de una imagen.

Dockerfile: Archivo de instrucciones que define cómo construir una imagen Docker.

Incluye pasos como: elegir la base, copiar el JAR, instalar dependencias y definir el comando de arranque.

docker load: Importa una imagen desde un archivo .tar. Ejemplo: docker load -i zeerty_app.tar

docker save: Exporta una imagen Docker a un archivo .tar para entregar. Ejemplo:

docker save -o zeerty_app.tar zeerty-app:latest

E - K

Endpoint: es una URL específica asociada a un método en una clase controladora (@RestController o @Controller) que recibe peticiones HTTP de un cliente y devuelve una respuesta.

Entity (Entidad): Clase Java que representa una tabla en la base de datos, marcada habitualmente con @Entity.

Fragmentos Thymeleaf: Componentes reutilizables que permiten incluir el mismo header y footer en todas las vistas del sistema mediante th:replace. Facilitan la coherencia visual y el mantenimiento.

Framework: Entorno de trabajo es una estructura conceptual y tecnológica de asistencia definida, normalmente, con artefactos o módulos concretos de software, que puede servir de base para la organización y desarrollo de software. Típicamente, puede incluir soporte de programas, bibliotecas, y un lenguaje interpretado, entre otras herramientas, para así ayudar a desarrollar y unir los diferentes componentes de un proyecto.

Frontend: Interfaz gráfica y visual con la que interactúa el usuario directamente en su dispositivo.

Gestión de Usuarios: Conjunto de funcionalidades que permiten al administrador crear, editar, eliminar y visualizar usuarios del sistema. Incluye validación de roles y control de acceso.

Idempotencia: es una propiedad en matemáticas, lógica e informática que permite aplicar una acción varias veces y obtener el mismo resultado que si se hiciera una sola vez.
GET, PUT y DELETE son idempotentes (repetir la petición deja el recurso en el mismo estado).

Imagen Docker (Docker Image): Plantilla inmutable que contiene todo lo necesario para ejecutar una aplicación: sistema base, dependencias, configuración y código.

Integridad de Datos: Propiedad que asegura que la información no ha sido alterada de forma indebida y que todas las operaciones quedan registradas correctamente.

Interfaz Web: Pantallas de la aplicación que se muestran en el navegador.

Inversión de Control (IoC): Principio por el cual el contenedor de Spring toma el control del flujo del programa y gestiona el ciclo de vida de los objetos.

Inyección de Dependencias (DI): Patrón de diseño donde el framework proporciona automáticamente los objetos que una clase necesita para funcionar, en lugar de crearlos manualmente.

JPA / Hibernate: Tecnologías para persistencia de datos:

Se integran mediante spring-boot-starter-data-jpa
JPA: especificación
Hibernate: implementación más usada

L - O

Login (inicio de sesión): es el proceso informático mediante el cual un usuario accede a una cuenta, sistema, programa o sitio web utilizando sus credenciales de identificación.

Log de Movimiento / Registro de Movimiento: Entrada almacenada en la base de datos que representa una acción realizada por un usuario sobre un activo digital. Incluye fecha, hora, tipo de acción, usuario y activo afectado.

Logout (o cierre de sesión) es el proceso mediante el cual un usuario finaliza una sesión autenticada en un sistema informático, una aplicación web o una cuenta en línea.
Maven / Gradle: Herramientas de construcción que gestionan dependencias, compilación y empaquetado.

Maven: declarativo, basado en XML.
Gradle: programable, rápido, usa Groovy/Kotlin DSL.

Modelo-vista-controlador (MVC): es un patrón de arquitectura de software, que separa los datos y principalmente lo que es la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones:

Componentes del MVC

- **Modelo:** Gestiona los datos, las reglas del negocio, las validaciones y la comunicación con la base de datos.
- **Vista:** Es la capa de presentación que muestra los datos al usuario en formato visual o HTML.
- **Controlador:** Actúa como intermediario; recibe las peticiones del usuario, consulta al modelo y selecciona la vista adecuada para responder.

Movimiento: Registro automático que se genera cuando un usuario crea, edita o elimina un activo. Sirve para llevar un historial de acciones.

P - S

Persistencia: Capacidad de la aplicación para guardar datos de forma permanente.

POJO (Plain Old Java Object): Objeto Java normal sin conocimiento de Spring. Los Beans pueden interactuar con POJOs.

Producto mínimo viable (MVP): versión básica de un nuevo producto o servicio que incluye solo las características esenciales para resolver el problema principal y satisfacer a los primeros clientes.

Prueba unitaria (o unit test): es una técnica de desarrollo de software que consiste en verificar que la unidad de código más pequeña de un programa (como una función, un método o una clase) funcione correctamente de forma aislada.

Red Docker (Docker Network): Red interna que permite que los contenedores se comuniquen entre sí usando nombres en lugar de IPs. Ejemplo: jdbc:mysql://mysql:3306.

Repository (Interfaz): Abstracción de Spring Data que proporciona métodos automáticos (como save, findAll, delete) para interactuar con la base de datos sin escribir SQL.

Rol: Permiso o nivel de acceso asignado a un usuario.

Ruta / Endpoint: Dirección interna que usa la aplicación para acceder a una funcionalidad. Ejemplo: /activos/lista

Sesión: Espacio temporal donde se almacena información del usuario autenticado. Se invalida al realizar logout para garantizar seguridad.

Spinner: en CSS es una animación gráfica (un indicador de carga o loader) que gira de forma continua para informar al usuario que la página web o una aplicación está procesando información, cargando datos o esperando una respuesta.

Spring Data JPA: Tecnología que permite interactuar con la base de datos sin escribir SQL manual, utilizando repositorios como LogMovimientoRepository.

Spring Boot: Framework de Java que simplifica el desarrollo de aplicaciones web y microservicios eliminando la mayor parte de la configuración manual.

Spring Boot Actuator: Módulo que añade funcionalidades para monitorear y gestionar la aplicación en producción (métricas, salud del sistema, etc.).

Starters: Dependencias preconfiguradas (ej. spring-boot-starter-web) que agrupan todas las librerías necesarias para una tarea específica.

T - Z

Trazabilidad: Capacidad del sistema para rastrear y reconstruir todas las acciones realizadas sobre los activos digitales, permitiendo conocer quién hizo qué y cuándo.

Toggle: interruptor de visibilidad en un formulario de login que sirve principalmente para alternar el campo de la contraseña entre texto oculto (type="password") y texto legible (type="text"), mejorando la experiencia del usuario al escribir.

Thymeleaf: Motor de plantillas utilizado para renderizar vistas HTML dinámicas en aplicaciones Spring Boot.

UML: (por sus siglas en inglés, Unified Modeling Language, o Lenguaje Unificado de Modelado) es un lenguaje gráfico estándar para visualizar, especificar, construir y documentar los componentes de un sistema de software y modelos de negocio.

Usuario: Persona que utiliza la aplicación.

Validación: Proceso que comprueba que los datos introducidos por el usuario son correctos antes de guardarlos.

Vista: Archivo HTML con Thymeleaf que muestra la información al usuario.

Anotaciones Principales de Spring Bott (Annotations)

- **@SpringBootApplication:** Anotación de inicio que combina tres funciones básicas: configuración automática, escaneo de componentes y definición de la clase de configuración.
- @**Autowired:** Marca un punto donde Spring debe inyectar automáticamente una dependencia (un Bean) en una clase.
- @**Component:** Anotación genérica para indicar que una clase es un componente gestionado por Spring.
- @**Configuration:** Indica que una clase define métodos productores de Beans usando la anotación @Bean.

Arquitectura en Capas (Stereotypes)

- **@RestController:** Combina @Controller y @ResponseBody. Indica que la clase manejará peticiones HTTP y devolverá los datos directamente en formato JSON o XML.
- **@Service:** Marca clases que contienen la lógica de negocio principal de la aplicación.
- @**Repository:** Indica que la clase interactúa con la base de datos (capa de acceso a datos) y traduce las excepciones de almacenamiento.

Desarrollo Web y APIs (Spring MVC)

- **@RequestMapping:** Enruta peticiones HTTP a métodos específicos del controlador basándose en la URL.
- **@GetMapping / @PostMapping:** Atajos específicos para manejar peticiones HTTP GET (lectura) y POST (creación).
- **@PathVariable:** Extrae variables directamente de la URL de la petición (ej. /usuarios/{id}).
- **@RequestBody:** Convierte automáticamente el cuerpo de la petición HTTP (normalmente JSON) en un objeto Java.

https://albertprofe.dev/learning/learning-vocab-spring-boot.html

https://www.linkedin.com/pulse/conceptos-springboot-juan-sebasti%C3%A1n-fern%C3%A1ndez-montoya-spvve/

https://www.devtools.pt/es/dev/cheatsheets/spring-boot